



Blazor Components: Fundamentals to Advanced Patterns



Andrea Dottor

andrea@dottor.net | Senior Developer & Consultant

Microsoft MVP Developer Technologies

Un grazie agli sponsor e alle community

studio{MIT}



Perché parlare di componenti nel 2026

I componenti sono diventati:

- l'unità base della UI
- il punto in cui vivono stato, logica e rendering
- il vero confine della complessità applicativa

Con Blazor questo è ancora più evidente:

- lo stesso linguaggio e lo stesso modello tra frontend e backend.

Componenti semplici vs sistemi complessi

All'inizio tutto è semplice:

- componenti piccoli, chiari, facili da gestire.

Poi l'app cresce:

- più stato da gestire
- più interazioni tra componenti
- più logica distribuita

E iniziano i problemi:

- componenti troppo “intelligenti”
- stato duplicato e incoerente
- comunicazione difficile tra parti diverse

Cos'è davvero un componente Blazor

Un componente Blazor è:

- una classe .NET (che eredita da ComponentBase)
- con stato interno
- che produce un render tree

Il rendering è automatico:

- cambia lo stato
- Blazor aggiorna la UI



Non aggiorni il DOM
aggiorni lo stato

Rendering model: quando e perché re-renderizza

Il rendering non è manuale, è guidato dallo stato.

Un componente re-renderizza quando:

- cambia un parametro
- viene eseguito un evento (es. click)
- chiami `StateHasChanged()`



Non tutto viene ridisegnato,
solo ciò che cambia

Data flow: parent → child → parent

Il flusso dei dati è unidirezionale:

- Parent → Child (parametri)
- Child → Parent (eventi)

Il child non modifica direttamente il parent ma notifica un cambiamento



I dati scendono,
gli eventi risalgono

StateHasChanged e ShouldRender: controllo minimo

StateHasChanged() → forza un aggiornamento
ShouldRender() → decide se aggiornare

Usali con cautela



Meno controllo manuale
più prevedibilità



PATTERN REALI

Presentation vs Container

Problema: componenti troppo intelligenti

Troppa logica dentro un componente

- UI + stato + regole insieme
- difficile da testare
- difficile da riutilizzare



Il componente diventa rigido
più fa, meno è riutilizzabile

Pattern: separare UI da logica

Separare responsabilità

- **UI** → presentazione
- **logica** → comportamento



Componenti più semplici
più riutilizzabili



Separare rende scalabile

Esempio: Container + Presentational component

il **Container** sa cosa succede (carica dati, gestisce stato, decide cosa fare)
il **Presentational** sa come mostrarlo

- 👉 **Container**
 - chiama API
 - tiene lo stato
 - decide quando aggiornare
- 👉 **Presentational**
 - mostra i dati
 - emette eventi (OnClick, OnChange)
 - non ha logica di business



Cambi UI senza toccare la logica



Controlled Components

Problema: stato duplicato e incoerente

Parent.razor

```
<h3>Cart</h3>
```

```
<CartSummary />
```

```
<CartTotal />
```

CartTotal.razor

```
<p>Total items: @_count</p>
```

```
@code {
```

```
    private int _count = 0;
```

```
}
```

CartSummary.razor

```
<p>Items: @_count</p>
```

```
<button @onclick="Add">Add item</button>
```

```
@code {
```

```
    private int _count = 0;
```

```
    private void Add() => _count++;
```

```
}
```

Problema: stato duplicato e incoerente

Lo stesso stato vive in più componenti

- valori duplicati
- aggiornamenti non sincronizzati
- bug difficili da tracciare

💀 La UI può diventare incoerente

🏆 Una sola fonte di verità

Pattern: il parent controlla lo stato

```
<h3>Cart</h3>
```

```
<CartSummary Count="_count" OnAdd="AddItem" />
```

```
<CartTotal Count="_count" />
```

```
@code {  
    private int _count = 0;  
    private void AddItem()  
        => _count++;  
}
```

CartTotal.razor

```
<p>Total items: @Count</p>
```

```
@code {  
    [Parameter] public int Count { get; set; }  
}
```

CartSummary.razor

```
<p>Items: @Count</p>
```

```
<button @onclick="OnAdd">Add item</button>
```

```
@code {  
    [Parameter] public int Count { get; set; }  
    [Parameter] public EventCallback OnAdd { get; set; }  
}
```



Headless Components

Problema: componenti poco riutilizzabili

UI e logica accoppiate

- difficile personalizzare
- difficile riutilizzare

💀 Troppi vincoli

Pattern: logica senza UI (RenderFragment)

Il componente espone:

- stato
- comportamento

La UI è delegata



Massima flessibilità

Un headless component non è un componente senza markup.
È un componente che incapsula comportamento, ma delega il rendering.

Esempio: componente headless

- RenderFragment + context
- Il consumer decide il rendering
- Il componente non impone la UI



Cascading Context

DashboardPage.razor

```
<PageHeader Theme="_theme" />
```

```
@code {  
    private ThemeInfo _theme = new("dark", "bg-dark text-white");
```

PageHeader.razor

```
public render() {  
    <HeaderActions Theme="Theme" />
```

```
}
```

```
@code {  
    [Parameter, EditorRequired]  
    public DashboardPage.ThemeInfo Theme { get; set; } = default;  
}
```

HeaderActions.razor

```
<UserMenu Theme="Theme" />
```

```
@code {  
    [Parameter, EditorRequired]  
    public DashboardPage.ThemeInfo Theme { get; set; } = default;  
}
```

UserMenu.razor

```
<div class="@Theme.CssClass p-2">  
    Current theme: @Theme.Name  
</div>
```

```
@code {  
    [Parameter, EditorRequired]  
    public DashboardPage.ThemeInfo Theme { get; set; }  
}
```

Problema: prop drilling

Passare dati lungo tutta la gerarchia

- codice verboso
- accoppiamento

💀 Poco scalabile

DashboardPage.razor

```
<CascadingValue Value="_theme">  
  <PageHeader />  
</CascadingValue>
```

```
@code {  
  private ThemeInfo _theme = new("dark", "bg-dark text-white");  
  
  public record ThemeInfo(string Name, string CssClass);  
}
```

PageHeader.razor

```
<HeaderActions />
```

HeaderActions.razor

```
<UserMenu />
```

UserMenu.razor

```
<div class="@Theme?.CssClass p-2">  
  Current theme: @Theme?.Name  
</div>
```

```
@code {  
  [CascadingParameter]  
  public DashboardPage.ThemeInfo? Theme { get; set; }  
}
```

Pattern: condividere stato con CascadingValue

CascadingValue / CascadingParameter

- accesso diretto ai dati
- meno passaggi
- meno accoppiamento

🏆 Stato condiviso in modo pulito



Component Communication

Problema: componenti non in gerarchia

Componenti scollegati

- nessun parent comune
- difficile comunicazione

💀 Soluzioni fragili

Pattern: state container / event bus leggero

State container / event pattern

- disaccoppiamento
- comunicazione fluida



I componenti non devono conoscersi

Form Integration Pattern

MyTextInput.razor

```
<input class="form-control"
      value="@Value"
      @oninput="OnInput" />
```

```
@code {
    [Parameter]
    public string? Value { get; set; }

    [Parameter]
    public EventCallback<string?> ValueChanged { get; set; }

    private async Task OnInput(ChangeEventArgs e)
    {
        Value = e.Value?.ToString();
        await ValueChanged.InvokeAsync(Value);
    }
}
```

Problema: input custom non integrati nei form

- Un componente custom può sembrare corretto:
 - aggiorna il valore (@bind funziona)
 - il modello viene aggiornato
- Ma NON è davvero integrato con EditForm
 - ✗ Cosa manca
 - ValueExpression
 - EditContext
 - NotifyFieldChanged
 - ⚠ Consequenze
 - validazione non coerente
 - stato del campo non aggiornato

Pattern: integrazione con EditContext

- Un input Blazor “vero” non aggiorna solo il valore
 - 👉 si integra con il contesto della form/EditForm
- Come funziona
 - recupera l'EditContext → CascadingParameter
 - identifica il campo → ValueExpression
 - notifica le modifiche → NotifyFieldChanged
- Risultato
 - validazione corretta
 - stato del campo aggiornato (modified / valid / invalid)
 - comportamento coerente con gli input nativi

re-render inutili

Problema: re-render inutili

- In Blazor il rendering è automatico e reattivo, ma non sempre è efficiente.
- Ogni cambiamento di stato può causare il re-render del componente e di tutta la sua subtree, anche quando non è realmente necessario.
- Questo porta a:
 - 💀 aggiornamenti inutili della UI
 - 💀 perdita di performance su componenti complessi
 - 💀 comportamenti difficili da prevedere

Soluzioni: granularità + key + evitare over-render

- Componenti più piccoli e focalizzati permettono a Blazor di aggiornare solo le parti realmente coinvolte, evitando di ridisegnare intere porzioni di UI.
- L'utilizzo della direttiva @key aiuta il framework a identificare correttamente gli elementi nelle liste, evitando ricostruzioni non necessarie e comportamenti inattesi.
- Inoltre, è importante evitare aggiornamenti di stato non necessari: meno cambiamenti significa meno render.

Esempio: re-render inutile

```
<button @onclick="Shuffle">Shuffle</button>
<ul>
  @foreach (var user in Users)
  {
    <li>@user.Name</li>
  }
</ul>

@code {
  private List<User> Users = new()
  {
    new("Andrea"),
    new("Luca"),
    new("Sara")
  };

  private void Shuffle()
  {
    Users = Users.OrderBy(x => Guid.NewGuid()).ToList();
  }

  public record User(string Name);
}
```

Soluzione: usare @key

```
<ul>
  @foreach (var user in Users)
  {
    <li @key="user">@user.Name</li>
  }
</ul>
```

- Con @key, Blazor sa che ogni elemento è identificato da user.
- Quindi:
 - 🏆 non ricrea tutto
 - 🏆 riusa gli elementi
 - 🏆 aggiorna solo ciò che serve"

Esempio: componente troppo grande

```
<h3>Counter: @count</h3>
<ul>
  @foreach (var user in Users)
  {
    <li>@user.Name</li>
  }
</ul>
<button @onclick="Increment">Increment</button>

@code {
  private int count = 0;

  private List<User> Users = new()
  {
    new("Andrea"),
    new("Luca"),
    new("Sara")
  };

  private void Increment() { count++; }
}
```

Soluzione: granularità

```
<Counter Count="count" OnIncrement="Increment" />
```

```
<ul>
  @foreach (var user in Users)
  {
    <li>@user.Name</li>
  }
</ul>
```

```
@code {
  private int count = 0;
  private List<User> Users = new()
  {
    new("Andrea"),
    new("Luca"),
    new("Sara")
  };

  private void Increment()
  {
    count++;
  }
}
```

Counter.razor

```
<h3>Counter: @Count</h3>
<button @onclick="Increment">Increment</button>
```

```
@code {
  [Parameter] public int Count { get; set; }
  [Parameter] public EventCallback OnIncrement { get; set; }

  private Task Increment() => OnIncrement.InvokeAsync();
}
```


5 regole per progettare componenti solidi

- Chi possiede lo stato?
 - Definiscilo chiaramente ed evita duplicazioni
- Parametri semplici e espliciti
 - Il componente deve avere un'API chiara
- Separare logica e UI
 - Evita componenti “tuttofare”
- Integrazione con il framework
 - Binding, EditContext, validazione
- Composizione prima di tutto
 - Preferire piccoli componenti riusabili



Domande?

Grazie!



@andreadottor

{

name:
email:
bsky:
linkedin:

}

Andrea Dottor
andrea@dottor.net
@andrea.dottor.net
@andreadottor



